

NiceGUI 中 app.urls 的详细解析

NiceGUI 的 `app.urls` 是用于**路由管理**的核心组件，本质是一个路由注册表，负责将 URL 路径与对应的 UI 处理逻辑（页面 / 组件 / 函数）绑定，实现 Web 应用的多页面 / 多路径导航。以下从核心特性、使用方式、高级用法等维度全面解析：

一、核心概念

`app` 是 NiceGUI 应用的核心实例（通常通过 `from nicegui import app` 导入），`app.urls` 是该实例下的路由映射对象，遵循**HTTP 路由的基本规则**：

- 键：URL 路径（字符串，如 `/`、`/user/{id}`）；
- 值：路由处理函数（可返回 UI 组件、页面布局，或执行业务逻辑）；
- 支持**路径参数**（如 `{id}`）、**通配符**（如 `/api/*`）、**HTTP 方法限定**（GET/POST 等）。

二、基础使用方式

1. 注册简单路由

通过 `app.urls.add()` 方法注册路由，最基础的用法是绑定路径与处理函数：

```
from nicegui import app, ui

# 注册根路径路由
@app.urls.add('/')
def index_page():
    ui.label('首页').classes('text-2xl')
    ui.button('跳转到关于页', on_click=lambda: ui.navigate.to('/about'))

# 注册/about路径路由
@app.urls.add('/about')
def about_page():
    ui.label('关于我们').classes('text-2xl')
    ui.button('返回首页', on_click=lambda: ui.navigate.to('/'))

ui.run()
```

- 访问 `http://localhost:8080/` 显示首页，`http://localhost:8080/about` 显示关于页；
- `ui.navigate.to()` 是 NiceGUI 内置的导航方法，依赖 `app.urls` 的路由配置。

2. 带路径参数的路由

支持动态路径参数（类似 FastAPI/Flask 的路径参数），参数会自动传入处理函数：

```
@app.urls.add('/user/{user_id}')
def user_detail(user_id: str): # 参数名需与路径中的{user_id}一致
    ui.label(f'用户ID: {user_id}').classes('text-x1')
    ui.button('返回首页', on_click=lambda: ui.navigate.to('/'))

# 访问http://localhost:8080/user/123, 会显示「用户ID: 123」
```

- 参数类型支持：字符串（默认）、整数（需手动转换，如 `int(user_id)`）；
- 多参数示例： `/order/{order_id}/item/{item_id}`。

3. 限定 HTTP 方法

默认路由响应 `GET` 请求，可通过 `methods` 参数指定支持的 HTTP 方法（如 `POST`、`PUT`）：

```
from fastapi import Request # NiceGUI基于FastAPI, 可直接使用FastAPI的Request

@app.urls.add('/api/login', methods=['POST'])
async def login_api(request: Request):
    data = await request.json() # 获取POST请求的JSON数据
    username = data.get('username')
    password = data.get('password')
    # 业务逻辑: 验证账号密码
    return {'code': 200, 'msg': '登录成功', 'data': {'username': username}}
```

- 支持的方法：`GET`、`POST`、`PUT`、`DELETE`、`PATCH` 等；
- 注意：处理 `POST` / `PUT` 等异步请求时，函数需加 `async`，并通过 `Request` 获取请求数据。

三、高级特性

1. 通配符路由（匹配任意子路径）

使用 `*` 作为通配符，匹配任意以指定前缀开头的路径，适用于动态路由或兜底路由：

```
# 兜底路由: 匹配所有未注册的路径
@app.urls.add('/{path:path}') # {path:path}是FastAPI的通配符语法
def not_found(path: str):
    ui.label(f'页面不存在: /{path}').classes('text-red-500 text-x1')
    ui.button('返回首页', on_click=lambda: ui.navigate.to('/'))
```

- 例如访问 `/abc/123`、`/test` 等未注册路径，都会触发该兜底路由；
- 注意：通配符路由需放在所有路由的最后注册（路由匹配遵循「先注册先匹配」原则）。

2. 路由依赖（中间件）

基于 FastAPI 的 `Depends`，可给路由添加前置依赖（如身份验证、参数校验）：

```
from fastapi import Depends
```

```
# 定义依赖函数：验证用户是否登录
def check_login():
    if not app.storage.user.get('is_login'): # NiceGUI的user storage存储用户状态
        ui.navigate.to('/login') # 未登录则跳转到登录页
        raise Exception('未登录') # 终止路由执行
    return True

# 需登录的路由：添加Depends(check_login)
@app.urls.add('/admin', dependencies=[Depends(check_login)])
def admin_page():
    ui.label('管理员页面').classes('text-xl')
    ui.button('退出登录', on_click=lambda: app.storage.user.update({'is_login': False}))
```

- 访问 `/admin` 时，会先执行 `check_login()`，未登录则跳转到登录页；
- 依赖函数可返回数据，路由函数可接收依赖的返回值（如 `def admin_page(login_status: bool = Depends(check_login))`）。

3. 移除已注册的路由

通过 `app.urls.remove()` 方法移除指定路径的路由（适用于动态调整路由）：

```
# 先注册路由
@app.urls.add('/temp')
def temp_page():
    ui.label('临时页面')

# 移除路由（参数为路径字符串）
app.urls.remove('/temp')
```

- 注意：移除路由后，访问该路径会触发兜底路由（若已注册）。

四、与 `@ui.page` 装饰器的关系

NiceGUI 提供了更简洁的 `@ui.page` 装饰器，本质是 `app.urls.add()` 的封装：

```
# 等价于 app.urls.add('/home')
@ui.page('/home')
def home_page():
    ui.label('Home Page')
```

- `@ui.page` 支持 `methods`、`dependencies` 等与 `app.urls.add()` 完全一致的参数；
- 区别：`app.urls.add()` 更灵活（可动态添加 / 移除路由），`@ui.page` 更简洁（适合静态页面）。

五、注意事项

1. 路由路径必须以 `/` 开头（如 `/user`，而非 `user`）；
2. 路由匹配区分大小写（如 `/User` 和 `/user` 是两个不同路由）；

3. 避免路由冲突：相同路径 + 相同 HTTP 方法的路由只能注册一次；
4. 静态文件路由：NiceGUI 默认将 `static` 目录映射到 `/static` 路径，无需手动注册。